Project Documentation

TITLE: Script-Controlled ACL – Restrict Record Access Based on Field Value

Team Details

Team ID:

SWTID-2026-6326

Team Size: 3

Team Leader: Saranya R

E-Mail ID: saranyaramar787@gmail.com

Team Member: mahalakshmi S

E-Mail ID: Magalakshmi18062007@gmail.com

Team Member: keshini M

E-Mail ID: dhkeshini@gmail.com

1.Introduction

Project Overview

ServiceNow is a cloud-based platform used to automate and manage enterprise IT service, operations, and business workflows. Security within ServiceNow is enforced primarily through Access Control Lists (ACLs), which determine who can create, read, write, or delete records and fields across the platform.

This project demonstrates how to implement a script-controlled ACL to restrict access to records based on the value of a specific field, rather than relying only on static roles. This approach allows access decisions to be made dynamically at runtime, based on the actual data in each record. **Objective**

- To understand the working of Access Control Lists (ACLs) in ServiceNow.
- To design and implement a script-controlled ACL that evaluates a field's value before granting record access.
- To restrict read/write access to records dynamically, depending on business logic rather than fixed roles alone.
- To test and validate that unauthorized users cannot view or edit restricted records.

Scope

The ACL is applied at the record (table) level and uses an Advanced script condition to evaluate a chosen field (for example, a status, category, or ownership field) at the time a user attempts to access a record. If the script returns true, access is granted; if it returns false, access is denied.

2. Understanding Access Control Lists (ACL)

An Access Control List (ACL) rule in ServiceNow secures a specific object (a table, field, or UI page) and operation (create, read, write, delete). Each rule can require a role, a condition, a script, or a combination of these before granting access.

Types of ACL Enforcement

- Role-based ACL – Access is granted only to users holding a specific role.
- Condition-based ACL – Access is granted when a simple field condition is met.
- Script-based (Advanced) ACL – Access is determined by evaluating a server-side script, allowing complex, dynamic logic.

This project uses the script-based (Advanced) approach, since the access decision depends on comparing the logged-in user against the value stored in a record field.

ACL Evaluation Order

1. Field-level ACLs are evaluated first.
2. Table-level (record) ACLs are evaluated next.
3. A user must pass every applicable ACL — role, condition, and script — to be granted access. Failing any one of them denies access.

3. Problem Statement / Scenario

Consider an Incident (or custom) table where each record has an “Assigned To” field and a “Watch List” field. The requirement is:

- A user should be able to read or edit a record only if they are the Assigned To user, or they are listed in the Watch List for that record.
- All other users, even if they can normally see the table, should be denied access to that specific record. Users
- with the admin role should retain full access regardless of the field value.

A simple role-based ACL cannot express this, because the decision depends on the contents of each individual record. A script-controlled ACL solves this by evaluating the field value for every access attempt.

4. Implementation Steps

4.1 Prerequisites

- Administrator access to a ServiceNow personal developer instance (PDI).
- The “security_admin” role, required to create or modify ACL rules.
- A target table (for example, Incident) with the relevant fields already present.

4.2 Steps to Create the Script-Controlled ACL

1. Navigate to System Security > Access Control (ACL).
2. Click New to create a new ACL rule.
3. Set Type to record, and select the target table (for example, Incident) as the Name.
4. Set Operation to read (repeat the process separately for write, if edit access should also be restricted).

5. In the Requires role related list, leave it empty (or add the base role needed to see the table at all) — the finegrained restriction will be handled by the script.
6. Check the Admin overrides box if administrators should bypass this restriction.
7. Open the Advanced tab and enable Script.
8. Enter the access-control script (see Section 4.3) in the Script field.
9. Save the ACL rule.

4.3 Sample ACL Script

```
answer = (function evaluateAccess() {    // Admins always have access when 'Admin overrides' is checked    if (gs.hasRole('admin')) {       return true;    }    var currentUserId = gs.getUserID();    var assignedTo = current.getValue('assigned_to');    var watchList  = current.getValue('watch_list');    if (assignedTo == currentUserId) {       return true;    }    if (watchList && watchList.indexOf(currentUserId) != -1) {       return true;    }    return false;})();
```

The script compares the current logged-in user's sys_id against the value stored in the record's “Assigned To” and “Watch List” fields. Access is granted only when a match is found, or denied by default otherwise.

4.4 Restricting a Specific Field (Optional Enhancement)

To additionally restrict editing of a single sensitive field (for example, an email or salary field) rather than the whole record, a field-level ACL can be created with Type set to record, and Name set to the table and field (for example, incident.email). The script for such a field ACL typically defaults to false and grants access only under an explicit condition, such as the user holding the admin role.

5. Testing and Validation

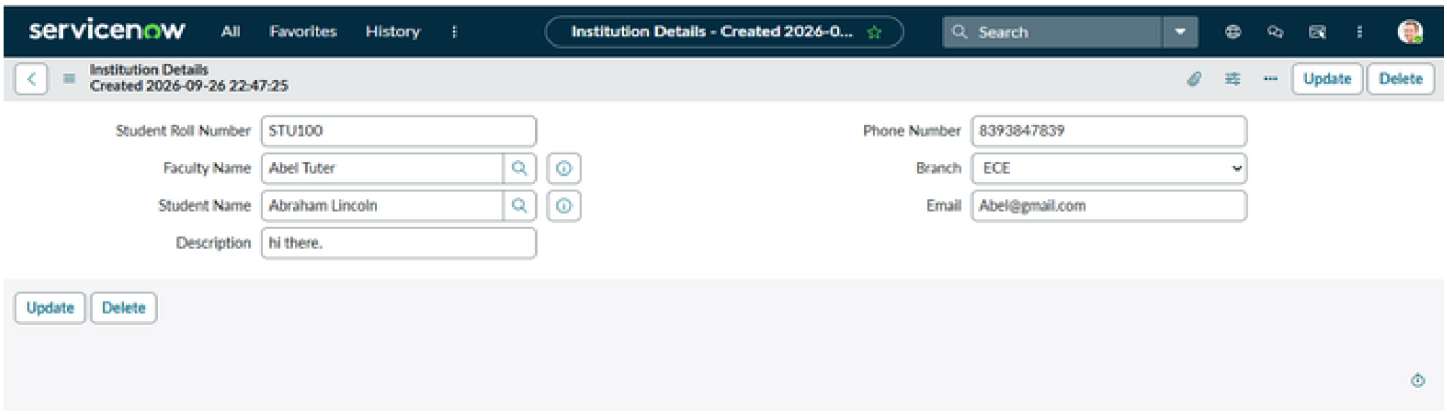
Test Cases

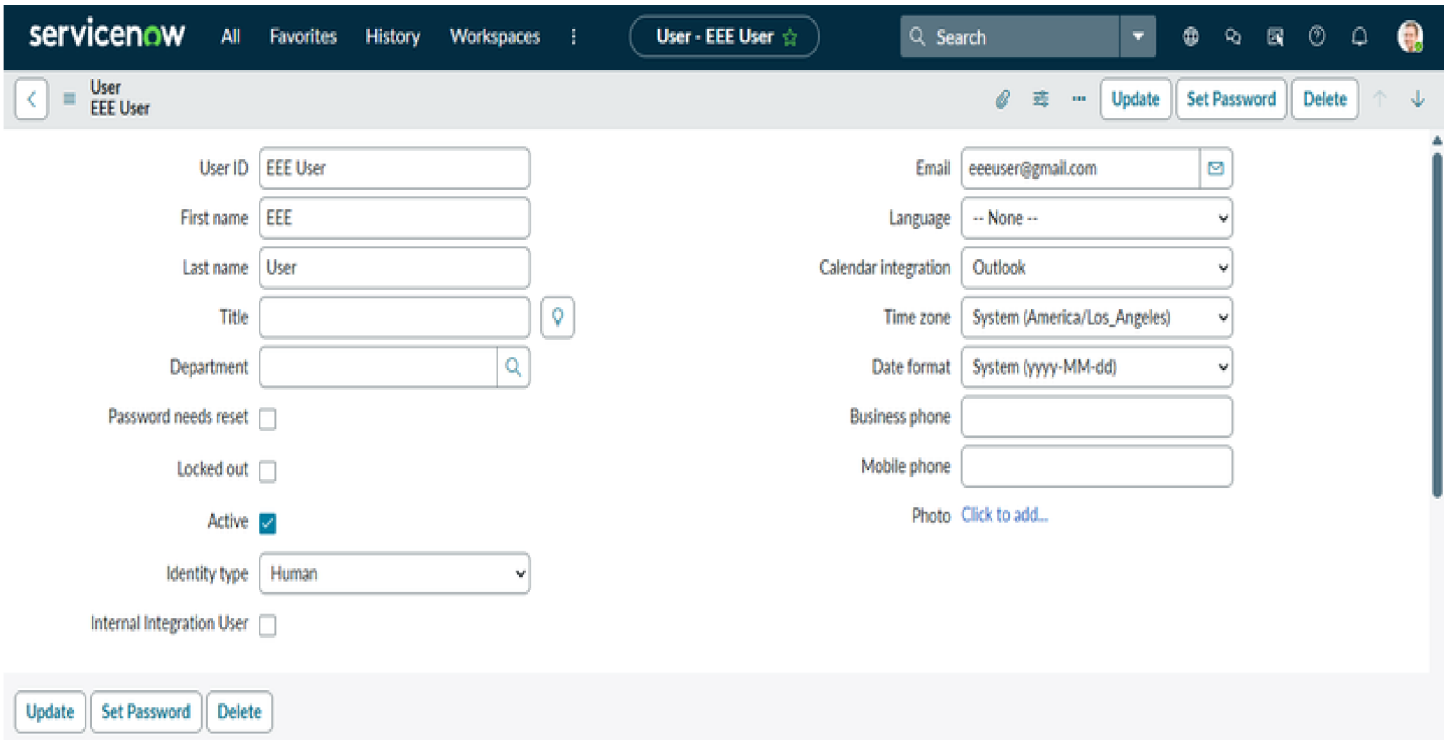
1. Log in as the user set as “Assigned To” on a record → the record should be visible and editable.
2. Log in as a user listed in the record's “Watch List” → the record should be visible.
3. Log in as a user who is neither the assignee nor on the watch list → the record should not appear in lists, forms, or search results.
4. Log in as an administrator → all records should remain accessible, confirming the Admin overrides setting works as intended.

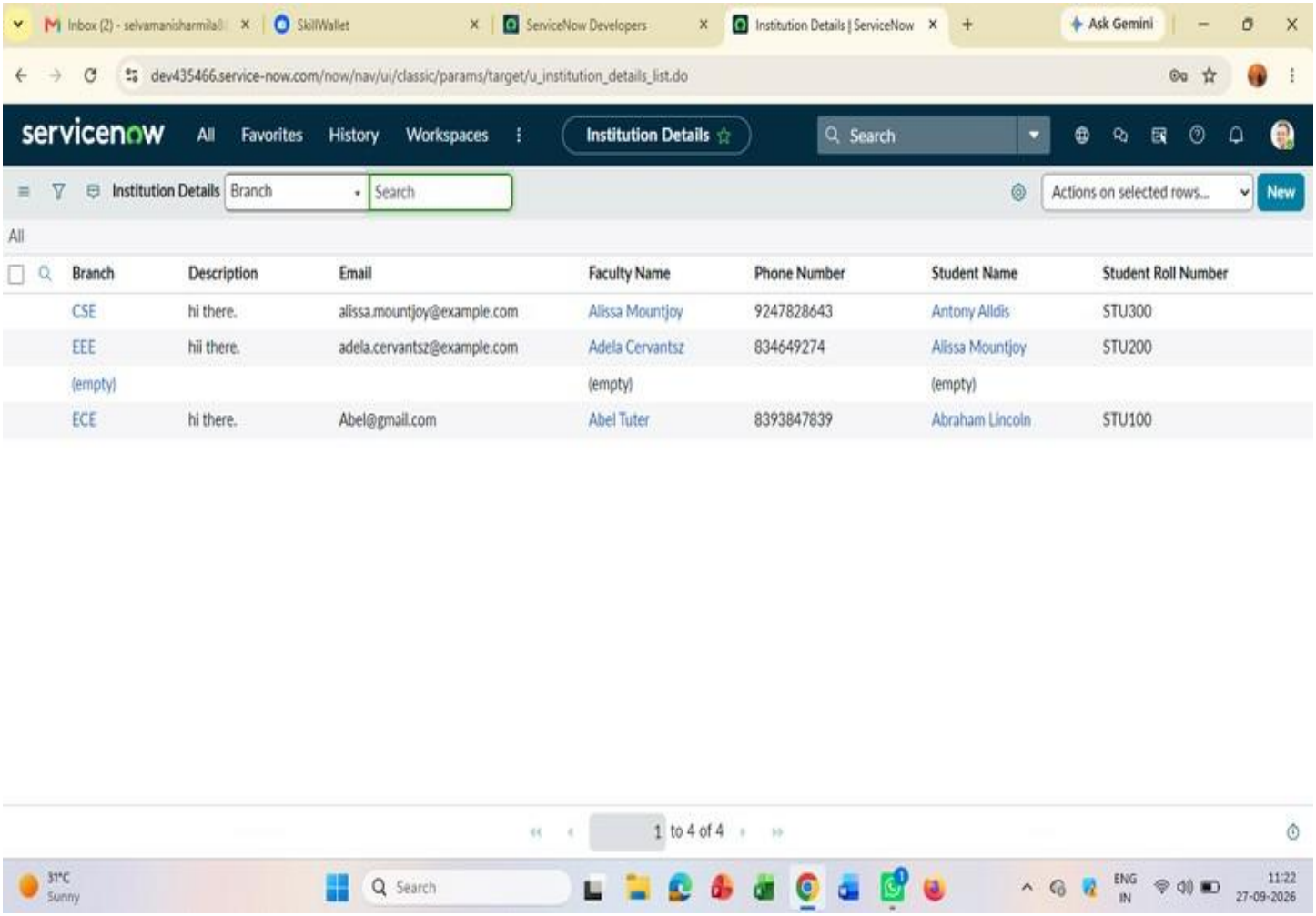
Observed Result

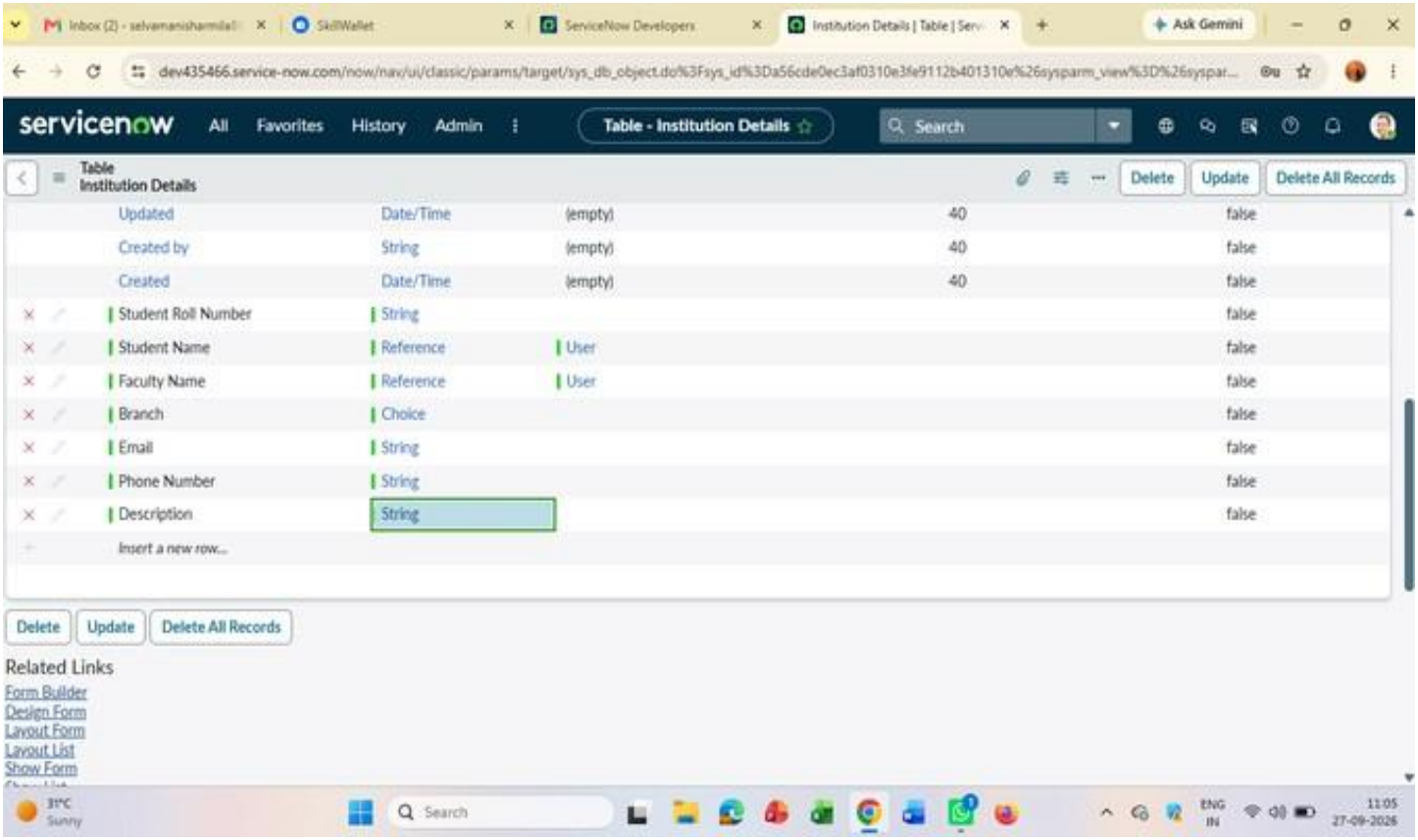
Testing confirmed that users outside the Assigned To / Watch List condition could not view or edit the restricted records through the form view, list view, or global search, while authorized users and administrators retained normal access. This validated that the script-controlled ACL correctly enforced record-level security based on field value.

6.Project Completion Proof


servicenow
All
Favorites
History
Institution Details - Created 2026-0...
Search
Institution Details
Created 2026-09-26 22:47:25
Update
Delete
Student Roll Number
STU100
Faculty Name
Abel Tuter
Student Name
Abraham Lincoln
Description
hi there.
Phone Number
8393847839
Branch
ECE
Email
Abel@gmail.com
Update
Delete

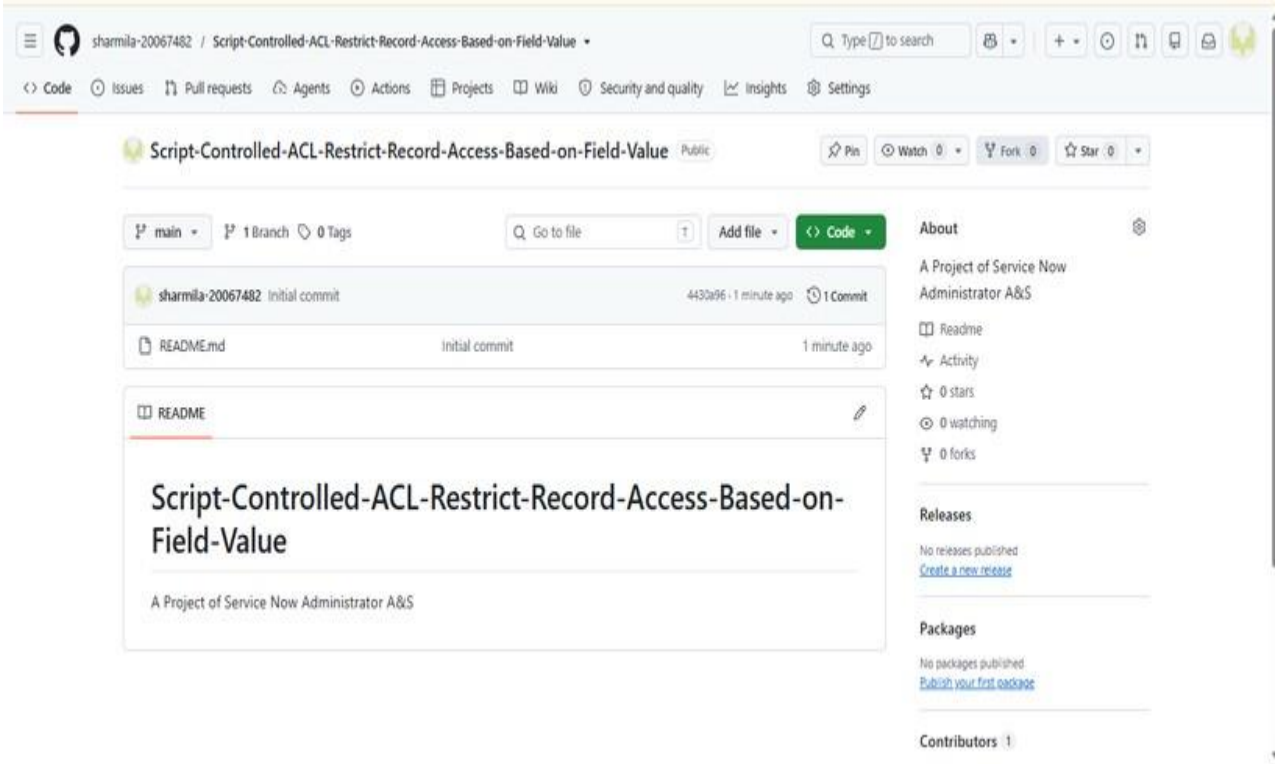

servicenow
All
Favorites
History
Workspaces
User - EEE User
Search
User
EEE User
Update
Set Password
Delete
User ID
EEE User
First name
EEE
Last name
User
Title
Department
Password needs reset
Locked out
Active
Identity type
Human
Internal Integration User
Email
eeeuser@gmail.com
Language
-- None --
Calendar integration
Outlook
Time zone
System (America/Los_Angeles)
Date format
System (yyyy-MM-dd)
Business phone
Mobile phone
Photo
Click to add...
Update
Set Password
Delete


Inbox (2) - selvamanisharmila
SkillWallet
ServiceNow Developers
Institution Details | ServiceNow
Ask Gemini
dev435466.service-now.com/now/nav/ui/classic/params/target/u_institution_details_list.do
servicenow
All
Favorites
History
Workspaces
Institution Details
Search
Institution Details
Branch
Search
Actions on selected rows...
New
All
Branch
Description
Email
Faculty Name
Phone Number
Student Name
Student Roll Number
CSE
hi there.
alissa.mountjoy@example.com
Alissa Mountjoy
9247828643
Antony Alldis
STU300
EEE
hii there.
adela.cervantsz@example.com
Adela Cervantsz
834649274
Alissa Mountjoy
STU200
(empty)
(empty)
(empty)
ECE
hi there.
Abel@gmail.com
Abel Tuter
8393847839
Abraham Lincoln
STU100
1 to 4 of 4
31°C
Sunny
Search
ENG
IN
11:22
27-09-2026



9.Git Hub Account

GitHub Link: https://github.com/sharmila-20067482/Script-Controlled-ACL-Restrict-Record-Access-Basedon-Field-Value.git



8. Conclusion

This project demonstrated how ServiceNow's Access Control List framework can be extended beyond static roles by using an Advanced script condition. By evaluating field values such as “Assigned To” and “Watch List” at runtime, record-level access can be restricted dynamically and accurately, ensuring that sensitive records are visible only to the users who are meant to see them. This pattern can be adapted to any table and field combination where access needs to depend on data rather than role alone.

Key Learnings

- ACL rules are evaluated in a strict order: field-level rules before table-level rules.
- Advanced (scripted) ACLs allow dynamic, data-driven access decisions that condition-only ACLs cannot express.
- The 'Admin overrides' setting is important to avoid accidentally locking out administrators.
- Thorough testing with multiple user roles is essential to confirm an ACL behaves as intended in every case.